PROJECT REPORT

Quotes Recommendation Chatbot using Rasa Prepared on: 2026-03-06

1. PREREQUISITES

Prerequisites are the requirements or conditions that must be fulfilled before starting something. They include prior knowledge, skills, and tools that ensure readiness and successful implementation. Project prerequisites: - Python 3.10 - Basic Python and YAML understanding - Rasa 3.6.20 and Rasa SDK 3.6.2 - Linux/WSL/macOS terminal and browser - Multi-terminal execution for action server, Rasa server, and frontend.

2. WORKFLOW

End-to-end workflow used in this project: 1) Define problem and business requirements 2) Set up environment and dependencies 3) Collect NLU training data and configure conversation logic 4) Train model and store generated artifacts 5) Test using shell, test stories, and smoke checks 6) Deploy with REST API + web frontend and validate behavior Milestone mapping: - Milestone 1: Define Problem / Problem Understanding - Milestone 2: Environment Setup - Milestone 3: Data Collection and Model Building - Milestone 4: Testing and Deployment.

3. DEFINE PROBLEM

Business problem: Users need quick motivational or emotional reassurance content, but manual searching is slow and inconsistent. Business requirements: - Understand natural language requests for quotes - Detect intent plus category/emotion entities - Recommend relevant quotes from supported categories - Ask follow-up feedback after quote delivery

Literature survey summary:

- Rule-based bots are predictable but rigid - Intent/entity-based assistants improve contextual relevance - Rasa provides customizable open-source conversational AI Social/business impact: - Supports positivity, self-help, and wellbeing - Useful for students, professionals, and general users - Extendable for wellness or engagement solutions

4. ENVIRONMENT SETUP

Dependencies and setup were standardized using a Makefile for reproducibility. Key setup commands: - make setup - make train Dependency stack: - rasa==3.6.20 - rasa-sdk==3.6.2 - setuptools>=68,<76 - sqlalchemy<2.0 Core configuration files: - config.yml (pipeline and policies) - domain.yml (intents, entities, slots, responses, actions) - endpoints.yml (action endpoint) - credentials.yml (REST channel)..

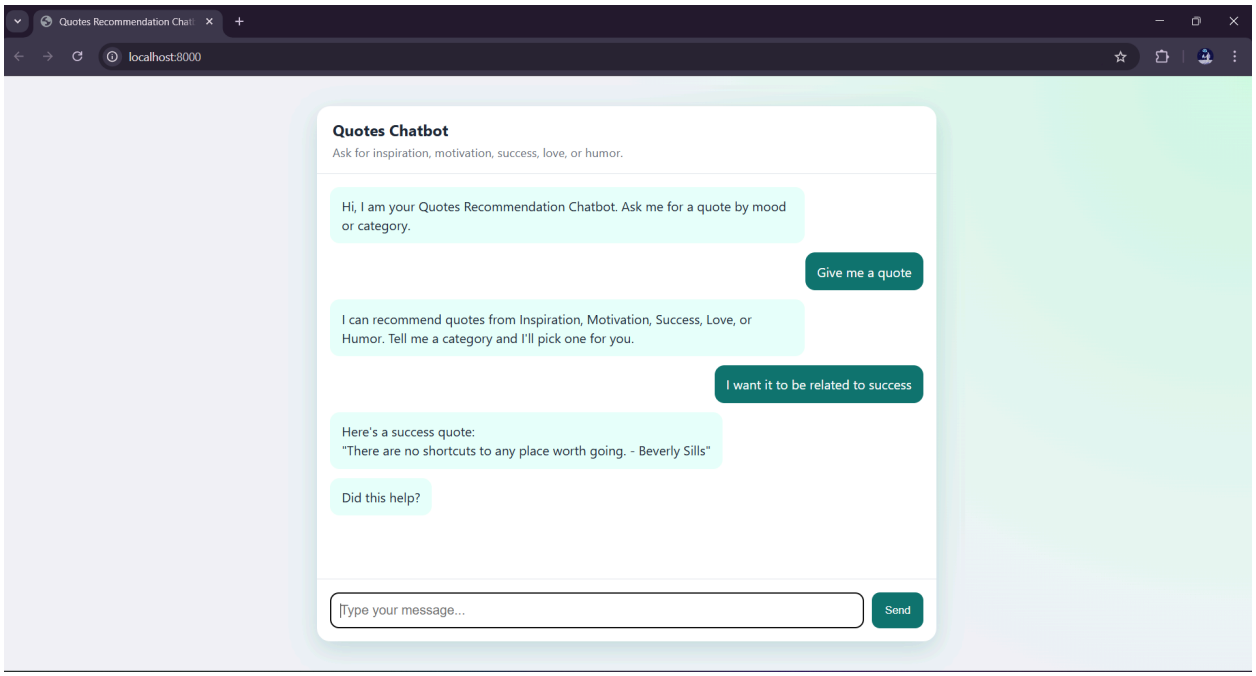
5. DATA COLLECTION AND MODEL BUILDING

Data collection and preparation: - Training examples created in data/nlu.yml for all key intents - Entities category and emotion annotated across examples - Synonyms, lookup, and regex included for robust extraction Conversation design: - Bot responses defined in domain.yml - Multi-turn interaction paths defined in data/stories.yml - Deterministic response rules defined in data/rules.yml Custom action implementation: - action_get_personalized_quote in actions/actions.py - Uses slots/entities/text cues to infer category and emotion - Maps emotion to category and returns a random quote from QUOTE_BANK Model training and storage: - Training command: make train

- Trained artifacts generated in models/ for reuse

6. TESTING AND DEPLOYMENT

Testing methods: - make shell for interactive behavior validation - make test for story-based validation - make smoke for REST webhook smoke testing Deployment steps (separate terminals): - make run-actions - make run-server - make run-frontend Deployment validation: - Frontend interaction at http://localhost:8000 - REST endpoint check at http://localhost:5005/webhooks/rest/webhook - Action health check at http://localhost:5055/health - Confirmed end-to-end personalized quote response flow.

7. CONCLUSION

Final Output